Homework 2: Rainfall in southern Italy

Spatial statistics 2026–2027 · Professor: Dr. Lasinio

Mojtaba Alinejad

21 October 2026

Contents

1 Data checks and choice of year **1**
1.1 What is actually in the two files? . . . 1
1.2 Correcting the Puglia label . . . 2
1.3 Why I chose 1970 . . . 3

2 Exploratory analysis **4**
2.1 Station map and rainfall distribution . . . 4
2.2 Relationships with elevation and location . . . 5
2.3 Trend, Gaussianity and the empirical variogram . . . 7

3 Choosing the IDW power with cross-validation **11**
3.1 A separate check of prediction performance . . . 13

4 Prediction on the supplied land grid **14**

5 Weak and good AI prompts **16**
5.1 Weak prompt . . . 16
5.2 Good prompt . . . 16

6 What I learned **17**

In this homework, I study rainfall in 1970. I first look at the station locations and the relationship between rainfall and elevation. Then I use cross-validation to choose the IDW power and make a rainfall map on the supplied grid. I explain my choices and discuss what I can learn from the results.

1 Data checks and choice of year

1.1 What is actually in the two files?

I load each file into its own environment. This avoids replacing my objects or random-number seed with objects stored in an R workspace.

```
rain_file <- new.env()
grid_file <- new.env()
invisible(load("data/RainData.RData", envir = rain_file))
invisible(load("data/AllGrids.RData", envir = grid_file))
rain_all <- rain_file$dati
land_grid <- rain_file$grid
marine_example <- grid_file$grid_1995

file_check <- data.frame(
  Object = c("RainData: dati", "RainData: grid", "AllGrids: grid_1995"),
```

```
  Rows = c(nrow(rain_all), nrow(land_grid), nrow(marine_example)),
  Content = c("Rainfall, 1970-1982", "Land grid with elevation",
              "Marine grid; other years 1996-2010")
)
knitr::kable(file_check)
```

Object	Rows	Content
RainData: dati	2373	Rainfall, 1970–1982
RainData: grid	1005	Land grid with elevation
AllGrids: grid_1995	517	Marine grid; other years 1996–2010

I found that **the two files do not belong to the same prediction task**. The objects in `AllGrids.RData` contain bathymetry, salinity and sea temperature, with years 1995–2010. Their coordinates are named `X` and `Y`; the rainfall file uses `XUTM` and `YUTM`. I cannot safely combine them just by changing column names or units. Their date ranges also do not overlap.

I found the matching land grid, `grid`, inside `RainData.RData`. It has 1,005 locations and their elevations. It joins `Basquota32` and `Calabriaquota32`. I use this existing grid, without creating a replacement regular grid. The objects `Bas` and `Cal` provide regional outline coordinates for the maps; `grid.int` is their concatenated outline, not the prediction grid.

```
expected_grid <- rbind(rain_file$Basquota32, rain_file$Calabriaquota32)
stopifnot(isTRUE(all.equal(land_grid, expected_grid,
                           check.attributes = FALSE)))
stopifnot(isTRUE(all.equal(rain_file$grid.int,
  rbind(rain_file$Bas, rain_file$Cal)[c("XUTM", "YUTM")],
  check.attributes = FALSE)))
stopifnot(!anyNA(land_grid),
          !anyDuplicated(land_grid[c("XUTM", "YUTM")]))
```

1.2 Correcting the Puglia label

The course hint identifies one Puglia label that should be Calabria. I check the three Puglia-labelled stations against the supplied Calabria outline. Only station **4367**, at approximately **38.55 N, 16.02 E**, is inside it. I correct that station's label across all years. I keep its coordinates, elevation and rainfall values unchanged. The other two Puglia-labelled stations remain in the dataset, and all stations in the chosen year enter the analysis.

```
station_info <- unique(rain_all[c("ID", "Nome_Regione", "Lat10",
                                  "Lon10", "XUTM", "YUTM")])
puglia <- subset(station_info, Nome_Regione == "Puglia")
puglia$inside_Calabria <- sp::point.in.polygon(
  puglia$XUTM, puglia$YUTM,
  rain_file$Cal$XUTM, rain_file$Cal$YUTM
) > 0
knitr::kable(puglia[c("ID", "Lat10", "Lon10", "inside_Calabria")],
             digits = 3)
```

	ID	Lat10	Lon10	inside_Calabria
5	4085	40.817	16.419	FALSE
6	4086	40.900	16.503	FALSE
150	4367	38.550	16.019	TRUE

```
wrong_id <- puglia$ID[puglia$inside_Calabria]
stopifnot(length(wrong_id) == 1, wrong_id == 4367)
```

```
rain_all$Nome_Regione <- as.character(rain_all$Nome_Regione)
fix_rows <- rain_all$ID == wrong_id
rain_all$Nome_Regione[fix_rows] <- "Calabria"
stopifnot(nrow(rain_all) == nrow(rain_file$dati),
          identical(rain_all$totanno, rain_file$dati$totanno))
```

This changes 13 annual labels, all belonging to the same station. I keep the original file unchanged. I do not change the label of every station near a regional boundary, because the supplied outline may not be exact.

1.3 Why I chose 1970

I choose the year with the largest number of complete station records. If years tie, I take the earliest. This gives a clear rule that does not depend on the rainfall pattern or prediction score. Complete annual records do not prove that every daily measurement used to calculate the annual total was available.

```
fields <- c("ID", "XUTM", "YUTM", "Quota", "totanno")
year_check <- do.call(rbind, lapply(sort(unique(rain_all$anno)),
  function(yr) {
    z <- subset(rain_all, anno == yr)
    data.frame(Year = yr, Stations = nrow(z),
               Complete = sum(complete.cases(z[fields])))
  }))
knitr::kable(year_check, row.names = FALSE)
```

Year	Stations	Complete
1970	189	189
1971	189	189
1972	187	187
1973	188	188
1974	187	187
1975	182	182
1976	181	181
1977	183	183
1978	182	182
1979	181	181
1980	178	178
1981	173	173
1982	173	173

```
chosen_year <- year_check$Year[which.max(year_check$Complete)]
dat <- subset(rain_all, anno == chosen_year)
stopifnot(chosen_year == 1970, !anyNA(dat),
          !anyDuplicated(dat$ID),
          !anyDuplicated(dat[c("XUTM", "YUTM")]),
          all(dat$totanno > 0), all(dat$Quota >= 0))
dat$rain <- dat$totanno
dat$x <- dat$XUTM / 1000
dat$y <- dat$YUTM / 1000
prediction_grid <- land_grid
prediction_grid$x <- prediction_grid$XUTM / 1000
prediction_grid$y <- prediction_grid$YUTM / 1000
```

1970 and 1971 tie with 189 complete records. I use **1970**, with **189 stations**. There are no missing values or duplicate locations in this subset. Other years are retained in the source file but are not pooled with 1970: repeated years at the same station are not independent spatial sites.

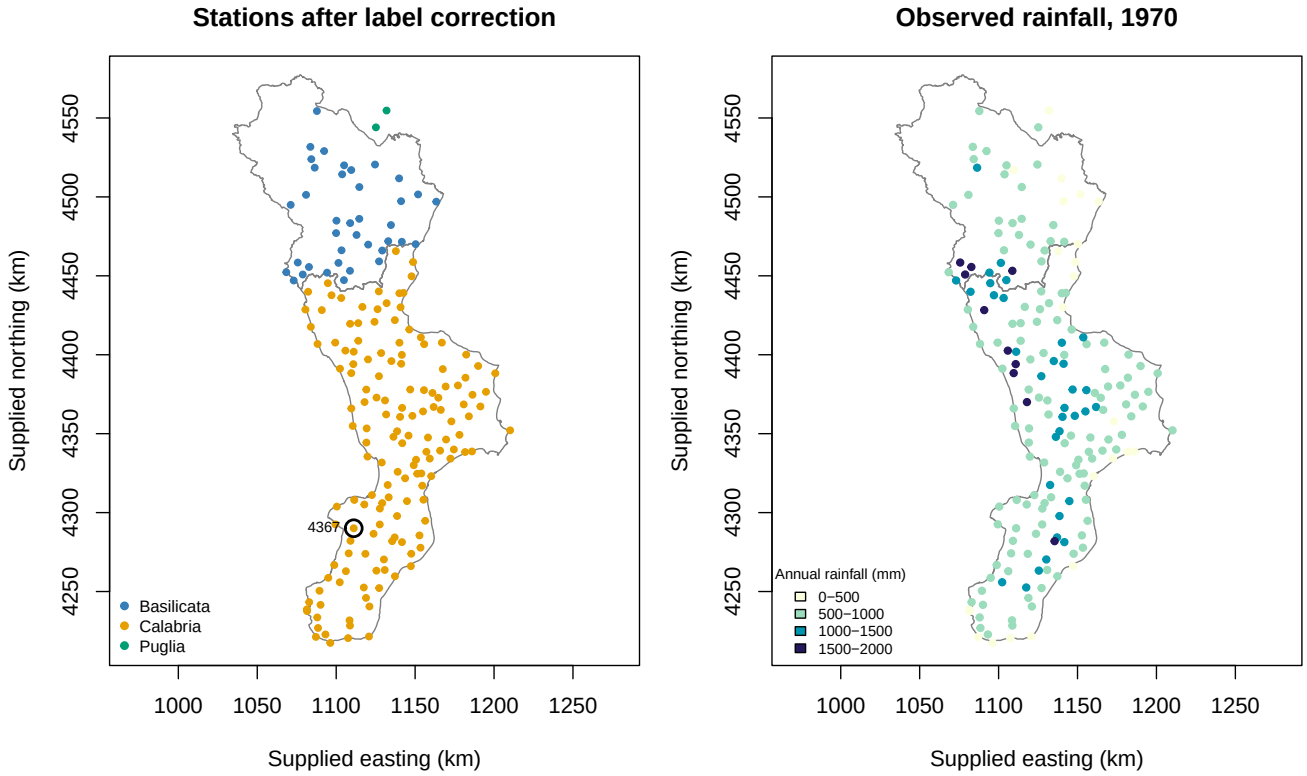
The response is `totanno`, the annual rainfall total. `Quota` is elevation; region is a geographical grouping, while coordinates describe location. I could not find a full description of the units or coordinate system in the files. I interpret rainfall as **mm**, elevation as **m**, and the supplied UTM-style coordinates as **m**, based on the variable names and the sizes of the values. I treat these units as assumptions because they are not stated in the files. I divide both coordinate axes by 1,000 for distances in km and retain their supplied common coordinate system. I do not assign an EPSG code without knowing the exact coordinate system. I calculate distances using the supplied projected coordinates. Changing both axes from metres to kilometres does not change the final IDW weights.

2 Exploratory analysis

2.1 Station map and rainfall distribution

```
region_col <- c(Basilicata = "#377EB8", Calabria = "#E69F00",
                Puglia = "#009E73")
draw_outlines <- function() {
  lines(rain_file$Bas$XUTM / 1000, rain_file$Bas$YUTM / 1000,
        col = "grey50")
  lines(rain_file$Cal$XUTM / 1000, rain_file$Cal$YUTM / 1000,
        col = "grey50")
}
map_frame <- function(title) {
  plot(prediction_grid$x, prediction_grid$y, type = "n", asp = 1,
       xlim = range(c(prediction_grid$x, dat$x)),
       ylim = range(c(prediction_grid$y, dat$y)),
       xlab = "Supplied easting (km)", ylab = "Supplied northing (km)",
       main = title)
  draw_outlines()
}
rain_breaks <- pretty(range(dat$rain), n = 5)
rain_col <- hcl.colors(length(rain_breaks) - 1, "YlGnBu", rev = TRUE)
rain_colour <- function(z) {
  rain_col[cut(z, breaks = rain_breaks, include.lowest = TRUE)]
}
rain_legend <- function() {
  labs <- paste(head(rain_breaks, -1), tail(rain_breaks, -1), sep = "-")
  legend("bottomleft", legend = labs, fill = rain_col,
         title = "Annual rainfall (mm)", cex = 0.65, bty = "n")
}
```

```
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
map_frame("Stations after label correction")
points(dat$x, dat$y, pch = 19, cex = 0.65,
       col = region_col[dat$Nome_Regione])
legend("bottomleft", legend = names(region_col), col = region_col,
       pch = 19, bty = "n", cex = 0.75)
i <- which(dat$ID == wrong_id)
points(dat$x[i], dat$y[i], pch = 1, cex = 1.7, lwd = 2)
text(dat$x[i], dat$y[i], "4367", pos = 2, cex = 0.7)
map_frame("Observed rainfall, 1970")
points(dat$x, dat$y, pch = 19, cex = 0.7, col = rain_colour(dat$rain))
rain_legend()
```



The stations cover an elongated area, with more stations in Calabria than in Basilicata. Their spacing is uneven. This matters because IDW uses nearby observations, so the information available for prediction changes across the map. The circled station is the corrected record, not a removed observation. Both outlines are supplied with the rainfall data; they are not a complete administrative map of Italy.

```
rain_summary <- data.frame(
  Minimum = min(dat$rain), Q1 = unname(quantile(dat$rain, 0.25)),
  Median = median(dat$rain), Mean = mean(dat$rain),
  Q3 = unname(quantile(dat$rain, 0.75)), Maximum = max(dat$rain),
  SD = sd(dat$rain)
)
knitr::kable(rain_summary, digits = 1)
```

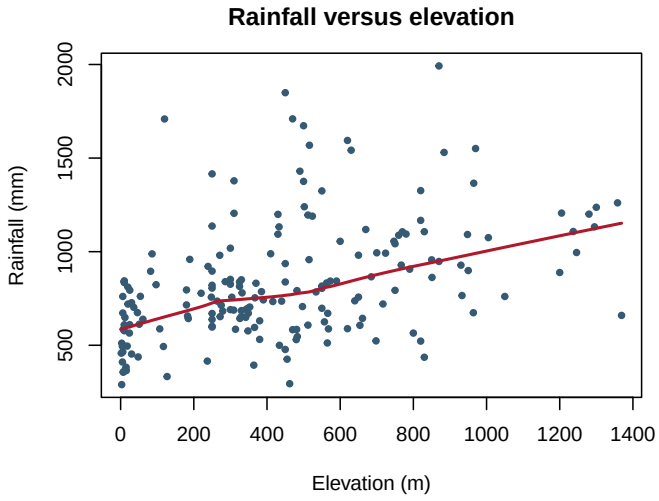
Minimum	Q1	Median	Mean	Q3	Maximum	SD
289.8	610.2	761.6	829.1	988.3	1992.2	317.3

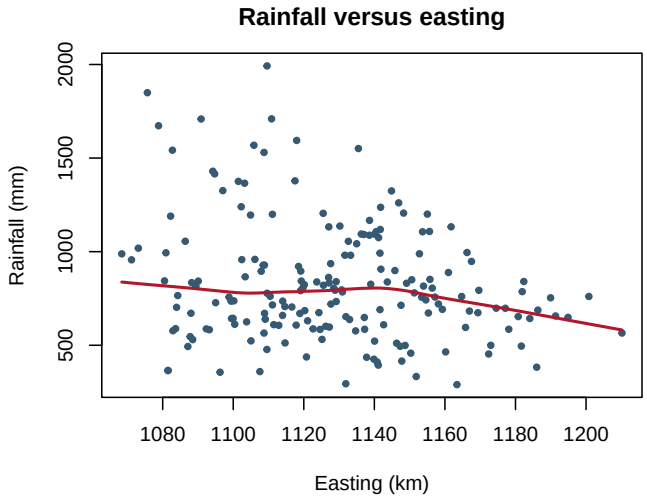
The mean is **829.1 mm** and the median is **761.6 mm**. The higher mean and the upper tail suggest right-skewness. I keep the large rainfall values: a high measurement alone is not evidence of a recording error.

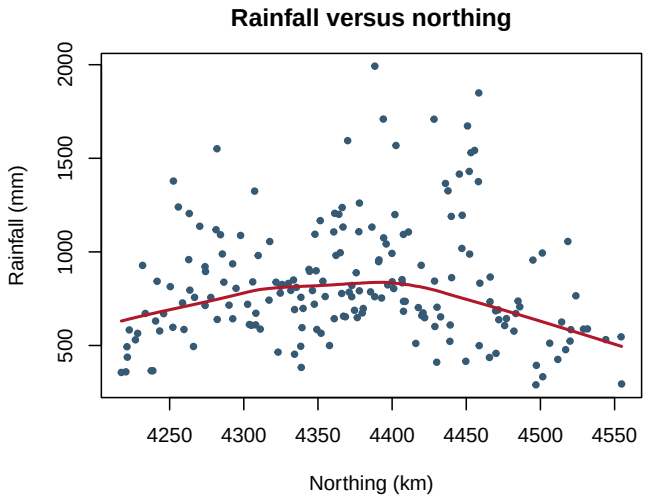
2.2 Relationships with elevation and location

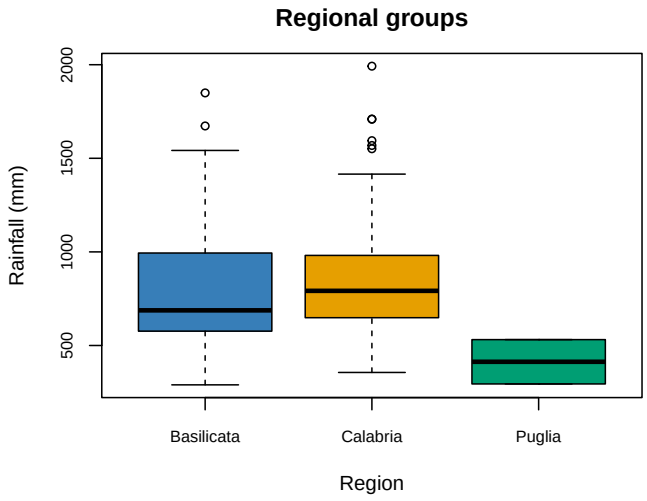
```
par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
for (v in c("Quota", "x", "y")) {
  axis_name <- c(Quota = "Elevation (m)", x = "Easting (km)",
                 y = "Northing (km)")[v]
  plot(dat[[v]], dat$rain, pch = 19, col = "#365A73", cex = 0.65,
       xlab = axis_name, ylab = "Rainfall (mm)",
       main = paste("Rainfall versus", tolower(sub(" \\(.*", "", axis_name))))
  lines(lowess(dat[[v]], dat$rain), col = "#B2182B", lwd = 2)
```

```
}
boxplot(rain ~ Nome_Regione, data = dat, col = region_col,
        ylab = "Rainfall (mm)", xlab = "Region", main = "Regional groups",
        cex.axis = 0.8)
```










```
cor_table <- data.frame(
  Descriptor = c("Elevation", "Easting", "Northing"),
  Pearson = sapply(dat[c("Quota", "x", "y")],
                   function(z) cor(z, dat$rain)),
  Spearman = sapply(dat[c("Quota", "x", "y")],
                    function(z) cor(z, dat$rain, method = "spearman"))
)
knitr::kable(cor_table, digits = 3, row.names = FALSE)
```

Descriptor	Pearson	Spearman
Elevation	0.432	0.476
Easting	-0.224	-0.147
Northing	0.019	-0.017

```
knitr::kable(as.data.frame(table(dat$Nome_Regione)),
             col.names = c("Region", "Stations"))
```

Region	Stations
Basilicata	38
Calabria	149
Puglia	2

I find a moderate positive relationship between rainfall and elevation (Pearson correlation **0.432**). Higher sites tend to receive more rain, but the points are quite spread out. Rainfall decreases somewhat with easting, while its simple linear association with northing is close to zero. This does not mean that northing is irrelevant after accounting for other variables, or that there is no nonlinear spatial pattern.

I use the regional boxplots to describe this sample. Only two stations remain labelled Puglia, so I cannot use that small group to describe rainfall across the whole region. These correlations do not show that one variable causes another. I would also be careful with ordinary correlation tests because nearby stations may be related.

2.3 Trend, Gaussianity and the empirical variogram

I compare three simple trend models: elevation alone, coordinates alone, and elevation plus coordinates. For each linear model, the leave-one-out residual is `residual / (1 - leverage)`. This gives its LOOCV RMSE without a long fitting loop. I use these trends to understand the data. For the final map, I still use ordinary IDW on rainfall rather than adding a regression model to IDW.

```
trend_models <- list(
  Elevation = lm(rain ~ Quota, data = dat),
  Coordinates = lm(rain ~ x + y, data = dat),
  Combined = lm(rain ~ Quota + x + y, data = dat)
)
cutoff <- max(dist(dat[c("x", "y")])) / 2
lag_width <- cutoff / 12
emp_variogram <- function(z) {
  vdat <- data.frame(x = dat$x, y = dat$y, value = z)
  gstat::variogram(value ~ 1, locations = ~ x + y, data = vdat,
                   cutoff = cutoff, width = lag_width)
}
residual_variograms <- lapply(trend_models,
                              function(m) emp_variogram(residuals(m)))
# I use the same exponential model and starting rule for each variogram.
# I fit these curves with weighted least squares to compare their nuggets.
nugget_fits <- lapply(seq_along(trend_models), function(j) {
  s2 <- var(residuals(trend_models[[j]]))
  gstat::fit.variogram(residual_variograms[[j]],
    gstat::vgm(psill = 0.8 * s2, model = "Exp",
               range = cutoff / 3, nugget = 0.2 * s2), fit.method = 7)
})
stopifnot(!any(vapply(nugget_fits, function(z)
  isTRUE(attr(z, "singular")), logical(1))))
trend_table <- data.frame(
  Trend = names(trend_models),
  CV_RMSE_mm = sapply(trend_models, function(m)
    sqrt(mean((residuals(m) / (1 - hatvalues(m)))^2))),
  Adjusted_R2 = sapply(trend_models, function(m) summary(m)$adj.r.squared),
  Nugget_mm2 = sapply(nugget_fits, function(v) v$psill[v$model == "Nug"])
)
knitr::kable(trend_table, digits = 2, row.names = FALSE)
```

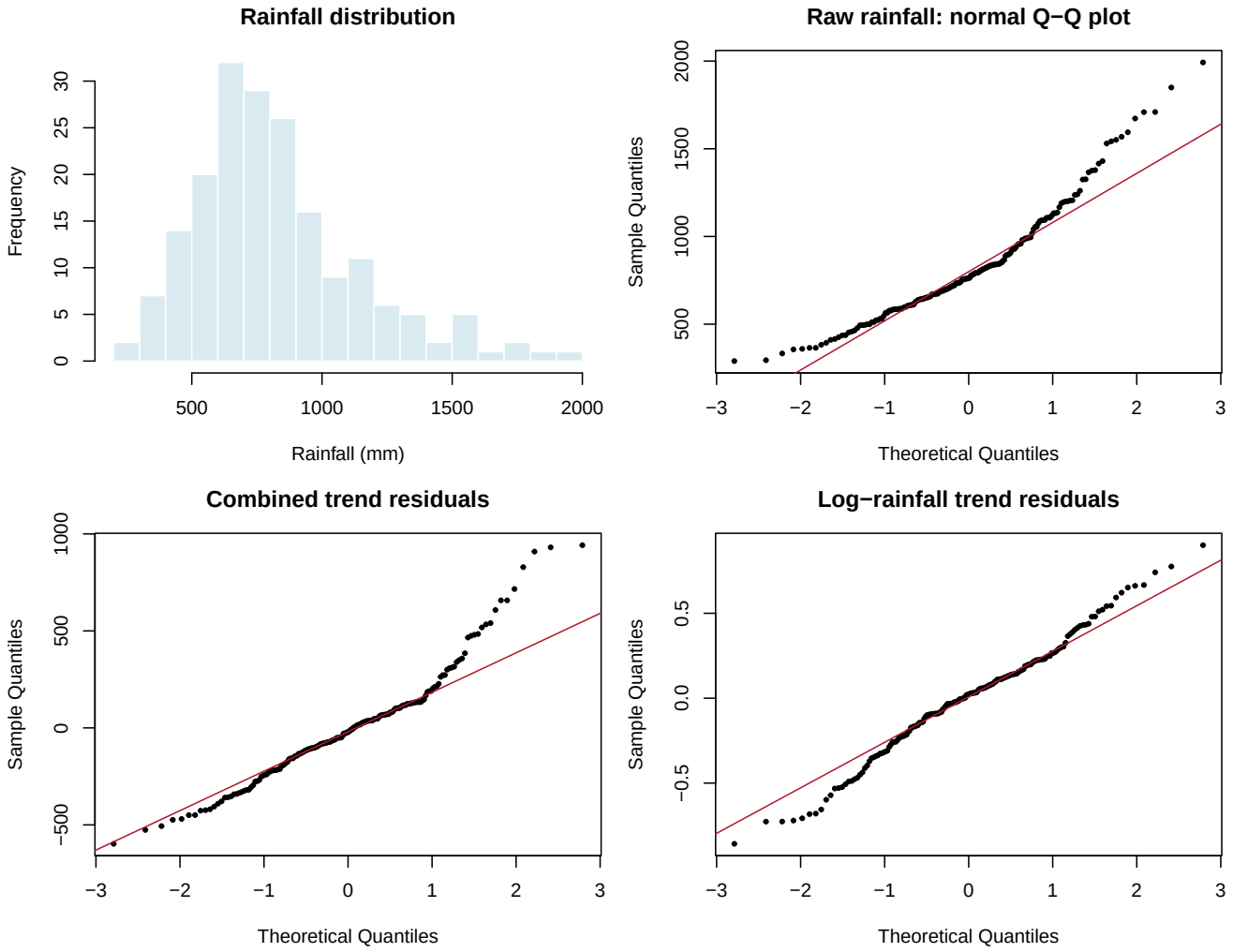
Trend	CV_RMSE_mm	Adjusted_R2	Nugget_mm2
Elevation	288.11	0.18	15325.45
Coordinates	313.64	0.04	19750.03
Combined	280.02	0.24	19882.48

```
trend <- trend_models$Combined
log_trend <- lm(log(rain) ~ Quota + x + y, data = dat)
```

The combined trend has the lowest CV RMSE among these three candidates (**280.02 mm**), but elevation alone has a lower fitted nugget. I therefore do not choose a trend by minimizing the nugget alone. I use the combined model for the next checks because it improves the prediction score and allows the mean to change across the area. However, it still leaves a lot of variation unexplained. Its adjusted R-squared is only **0.239**.

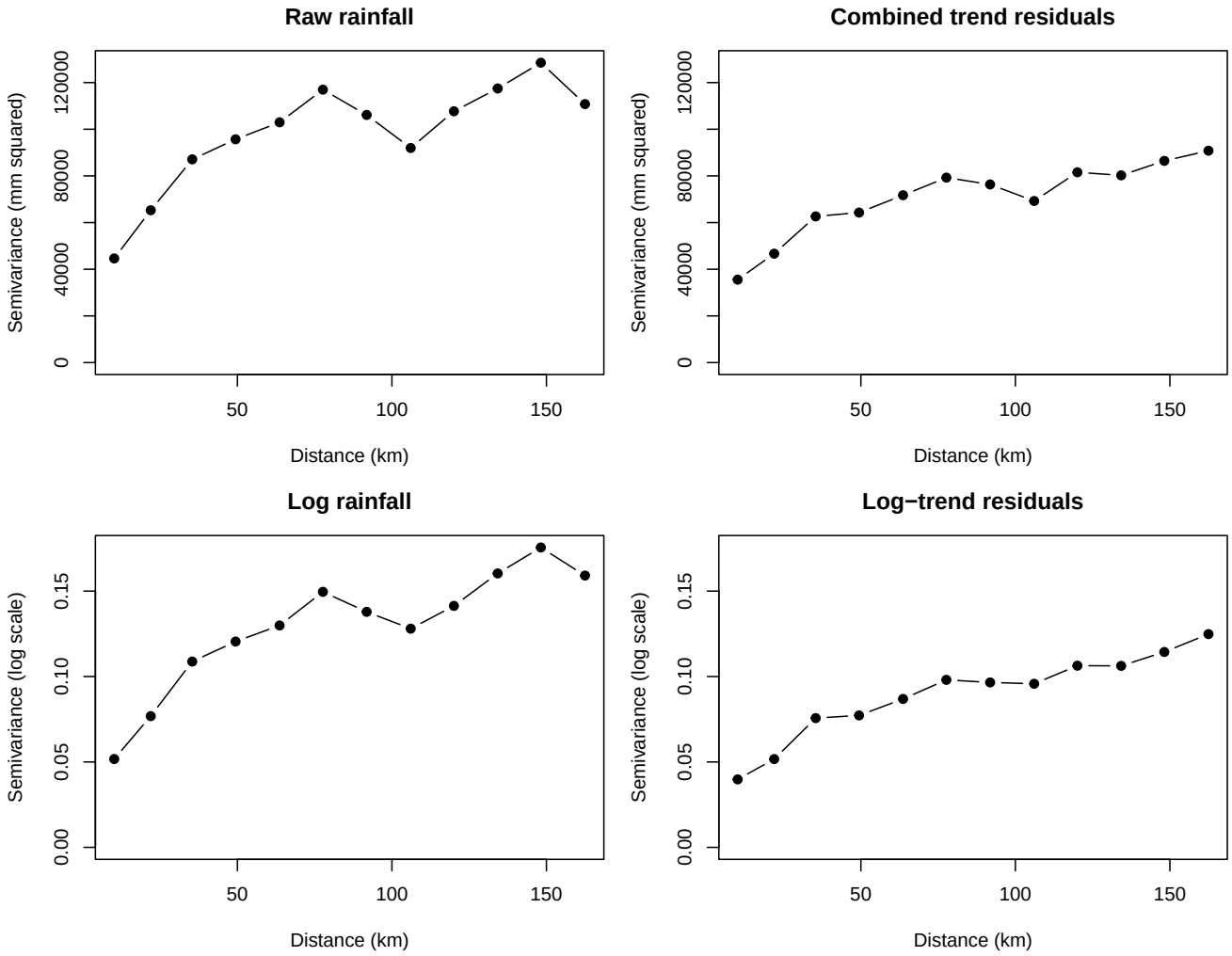
I read the nugget as the fitted semivariance at a distance close to zero. It can reflect measurement error and differences over distances smaller than those between my stations. I estimate it from the fitted curve, so it is not a direct measurement of error. It depends on the model, distance groups and trend. This is why I do not assume that the smallest nugget gives the best model. I also use the same data to compare these trends, so this comparison is not a separate test of the chosen trend.

```
par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
hist(dat$rain, breaks = "FD", col = "#D9EAF1", border = "white",
     main = "Rainfall distribution", xlab = "Rainfall (mm)")
qqnorm(dat$rain, main = "Raw rainfall: normal Q-Q plot", pch = 19, cex = 0.5)
qqline(dat$rain, col = "#B2182B")
qqnorm(residuals(trend), main = "Combined trend residuals",
       pch = 19, cex = 0.5)
qqline(residuals(trend), col = "#B2182B")
qqnorm(residuals(log_trend), main = "Log-rainfall trend residuals",
       pch = 19, cex = 0.5)
qqline(residuals(log_trend), col = "#B2182B")
```



In the Q-Q plots, I see that the raw rainfall and its trend residuals move away from a straight line. The log-rainfall residuals look closer to the line. Still, a normal-looking distribution by itself does not show that the whole spatial process is Gaussian. I use these plots as checks, rather than relying only on a normality test when nearby observations may be related.

```
v_raw <- emp_variogram(dat$rain)
v_res <- residual_variograms$Combined
v_log <- emp_variogram(log(dat$rain))
v_logres <- emp_variogram(residuals(log_trend))
par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
vv <- list(v_raw, v_res, v_log, v_logres)
titles <- c("Raw rainfall", "Combined trend residuals",
            "Log rainfall", "Log-trend residuals")
for (j in seq_along(vv)) {
  same_scale <- if (j <= 2) range(v_raw$gamma, v_res$gamma) else
    range(v_log$gamma, v_logres$gamma)
  plot(vv[[j]]$dist, vv[[j]]$gamma, type = "b", pch = 19,
       ylim = c(0, same_scale[2]), xlab = "Distance (km)",
       ylab = if (j <= 2) "Semivariance (mm squared)" else
         "Semivariance (log scale)", main = titles[j])
}
```




```
knitr::kable(data.frame(Lag = seq_len(nrow(v_raw)),
                        Mean_distance_km = v_raw$dist,
                        Pairs = v_raw$np,
                        Raw_semivariance = v_raw$gamma,
                        Residual_semivariance = v_res$gamma), digits = 2)
```

Lag	Mean_distance_km	Pairs	Raw_semivariance	Residual_semivariance
1	10.13	394	44625.76	35539.29
2	21.95	1019	65276.11	46668.12
3	35.39	1491	87086.77	62644.27
4	49.41	1648	95678.75	64281.95
5	63.65	1611	102970.67	71722.58
6	77.67	1529	116959.35	79264.50
7	91.83	1388	106131.45	76338.85
8	106.08	1273	91966.41	69247.34
9	120.07	1229	107698.45	81547.34
10	134.24	1087	117470.43	80240.54
11	148.19	980	128513.16	86435.85
12	162.49	874	110775.84	90797.14

These are 12 equal-width distance classes up to half the maximum station separation (**169.6 km**). I show the pair counts because a point based on fewer pairs is less stable. Removing the trend changes the variogram, but it does

not remove all variation with distance. The first residual bin has semivariance **35539.3**, compared with **44625.8 mm squared** for the raw observations.

A rising raw variogram may reflect a changing mean as well as spatial dependence. I use the residual plots to help separate these effects. They do not prove that the spatial behaviour stays the same everywhere (stationarity) or in every direction (isotropy). In the lower panels I repeat the check after taking logs. The units have changed, so I cannot directly compare their semivariance values with those in the upper panels.

I retain the original response scale for IDW because it estimates rainfall in its original units and does not require Gaussianity or a fitted covariance model. The log analysis above is diagnostic only. If I used log rainfall for prediction, I would need to apply that choice consistently during validation and mapping, and assess the predictions after appropriate back-transformation. Simply taking the exponential of a predicted log value does not usually give the mean rainfall on the original scale.

3 Choosing the IDW power with cross-validation

IDW predicts a weighted average of the observed rainfall. A station at distance d_i receives weight proportional to d_i^{-p}. A larger power puts more weight on nearby stations. I keep all stations as neighbours and tune only `p`, so the comparison has one changing setting.

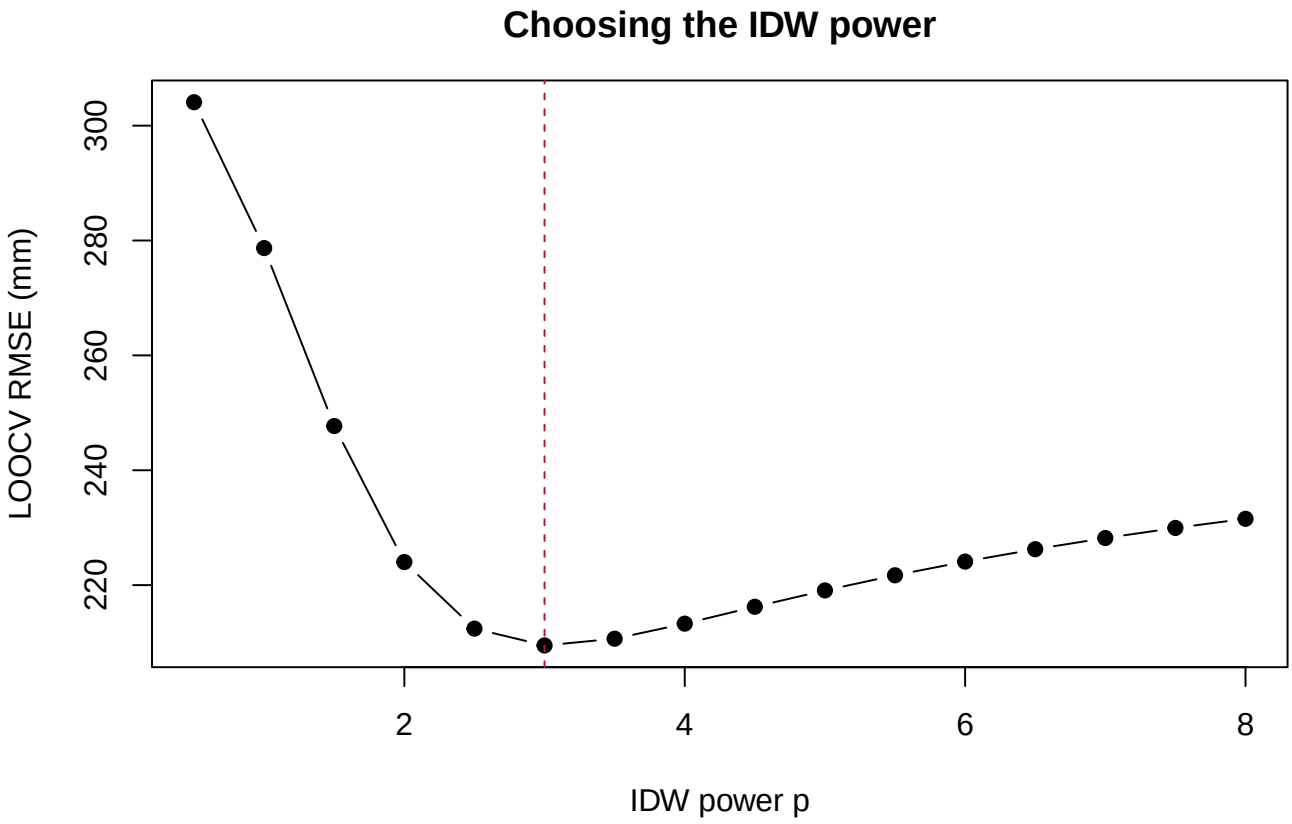
I test powers from 0.5 to 8 in steps of 0.5. Each observation is removed before predicting its value. The chosen power minimizes the leave-one-out RMSE. MAE and mean error provide additional information; mean error is prediction minus observation. The optimum is only the best among the candidates considered.

```
idw_predict <- function(train, newdata, p) {
  gstat::idw(rain ~ 1, locations = ~ x + y, data = train,
             newdata = newdata, idp = p, nmax = Inf,
             debug.level = 0)$var1.pred
}
score <- function(observed, predicted) {
  error <- predicted - observed
  c(RMSE = sqrt(mean(error^2)), MAE = mean(abs(error)), ME = mean(error))
}
choose_p <- function(train, powers = seq(0.5, 8, by = 0.5)) {
  predictions <- matrix(NA_real_, nrow(train), length(powers))
  for (j in seq_along(powers)) {
    for (i in seq_len(nrow(train))) {
      predictions[i, j] <- idw_predict(train[-i, ], train[i, ], powers[j])
    }
  }
  stopifnot(all(is.finite(predictions)))
  scores <- t(apply(predictions, 2, function(z) score(train$rain, z)))
  table <- data.frame(p = powers, scores, row.names = NULL)
  best <- which.min(table$RMSE)
  list(p = powers[best], table = table,
       predictions = predictions[, best])
}
tuning <- choose_p(dat)
best_p <- tuning$p
stopifnot(best_p > min(tuning$table$p), best_p < max(tuning$table$p))
knitr::kable(tuning$table, digits = 2)
```

p	RMSE	MAE	ME
0.5	304.08	234.47	10.43
1.0	278.68	216.08	15.51
1.5	247.70	192.12	16.66
2.0	224.02	172.78	17.63

p	RMSE	MAE	ME
2.5	212.43	161.30	18.56
3.0	209.49	155.90	18.91
3.5	210.67	154.37	18.84
4.0	213.29	154.42	18.62
4.5	216.23	154.77	18.40
5.0	219.08	155.47	18.23
5.5	221.71	156.43	18.10
6.0	224.10	157.45	18.02
6.5	226.25	158.45	17.96
7.0	228.20	159.31	17.92
7.5	229.96	160.06	17.89
8.0	231.55	160.69	17.87

```r
par(mfrow = c(1, 1), mar = c(4, 4, 3, 1))
plot(tuning$table$p, tuning$table$RMSE, type = "b", pch = 19,
     xlab = "IDW power p", ylab = "LOOCV RMSE (mm)",
     main = "Choosing the IDW power")
abline(v = best_p, lty = 2, col = "#B2182B")
```



The selected power is $\mathbf{p = 3}$, with tuning RMSE **209.49 mm**. For comparison, the common default `p = 2` gives **224.02 mm** under the same validation scheme. I also look at the nearby scores on the curve, because the selected value is the best of my tested values, not an exact physical constant.

3.1 A separate check of prediction performance

The minimum above was used to select p, so it is not an independent estimate of the selected procedure's accuracy. I add a simple outer five-fold check. For each outer training set, p is selected by LOOCV using only that training set. The untouched outer observations are then predicted. I use a training-mean baseline to check whether IDW improves on a simple non-spatial predictor.

```
set.seed(2026)
fold <- sample(rep(1:5, length.out = nrow(dat)))
outer_prediction <- baseline_prediction <- numeric(nrow(dat))
fold_p <- numeric(5)
for (k in 1:5) {
  test <- fold == k
  train <- dat[!test, ]
  inside <- choose_p(train)
  fold_p[k] <- inside$p
  outer_prediction[test] <- idw_predict(train, dat[test, ], inside$p)
  baseline_prediction[test] <- mean(train$rain)
}
outer_scores <- rbind(
  IDW = score(dat$rain, outer_prediction),
  Training_mean = score(dat$rain, baseline_prediction)
)
knitr::kable(outer_scores, digits = 2)
```

	RMSE	MAE	ME
IDW	228.78	167.53	24.23
Training_mean	320.64	244.30	-0.10

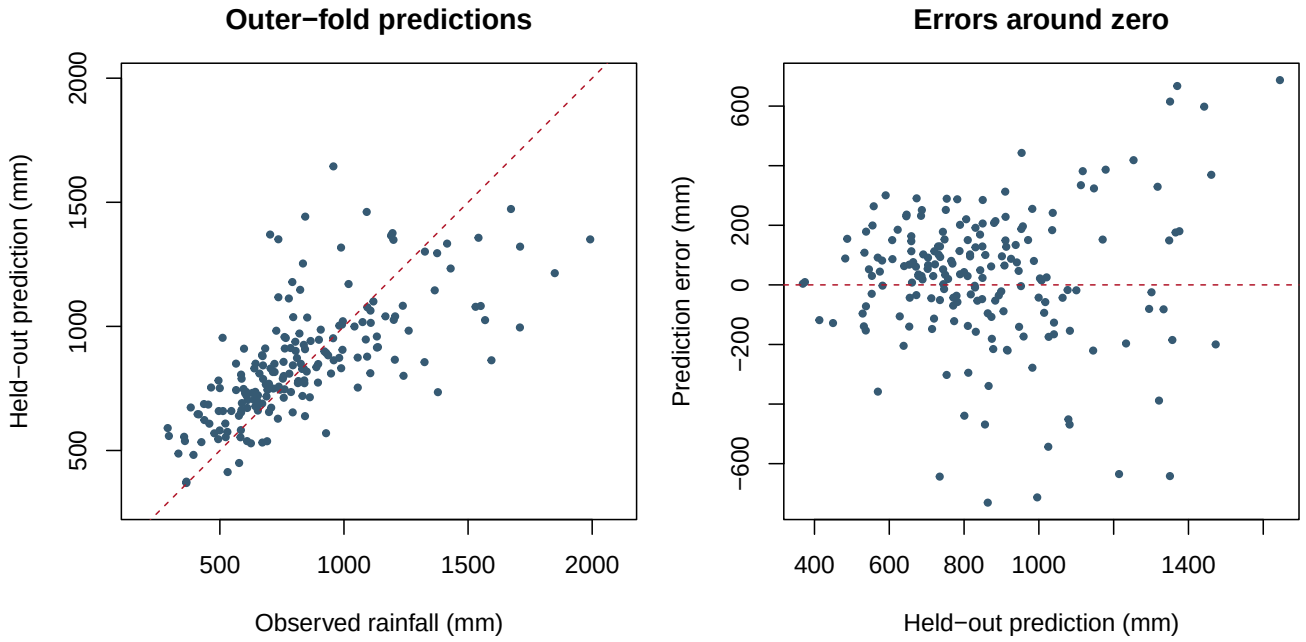
```
knitr::kable(data.frame(Fold = 1:5, Selected_p = fold_p))
```

Fold	Selected_p
1	3.0
2	2.5
3	3.0
4	3.0
5	3.0

The outer IDW RMSE is **228.78 mm**, and MAE is **167.53 mm**. These show how well I predict stations that were left out after choosing p inside each training fold. I use them to judge prediction accuracy rather than relying only on the lowest error from choosing p. The training-mean baseline has RMSE **320.64 mm**, so IDW reduces RMSE by about **28.6%** in this comparison. The mean IDW error is **24.23 mm**: predictions are slightly too high on average, even though some wet stations are underestimated. These scores are still based on one random partition, and use fewer training stations per fold than the final map. Differences between fold-specific powers show that the choice can depend on the available stations.

```
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
lim <- range(dat$rain, outer_prediction)
plot(dat$rain, outer_prediction, xlim = lim, ylim = lim, asp = 1,
     pch = 19, cex = 0.6, col = "#365A73",
     xlab = "Observed rainfall (mm)", ylab = "Held-out prediction (mm)",
     main = "Outer-fold predictions")
abline(0, 1, col = "#B2182B", lty = 2)
plot(outer_prediction, outer_prediction - dat$rain,
     pch = 19, cex = 0.6, col = "#365A73",
```

```
     xlab = "Held-out prediction (mm)", ylab = "Prediction error (mm)",
     main = "Errors around zero")
abline(h = 0, col = "#B2182B", lty = 2)
```



In the prediction plot, I can see how averaging nearby observations can smooth high and low rainfall values. A single RMSE does not show this clearly. With random folds, a nearby station may stay in the training data. This means my score is most useful for predicting between stations in this network. It may make prediction in large areas without stations look easier than it is. In further work, I could leave out whole spatial areas or repeat the random splits to see how much the results change.

4 Prediction on the supplied land grid

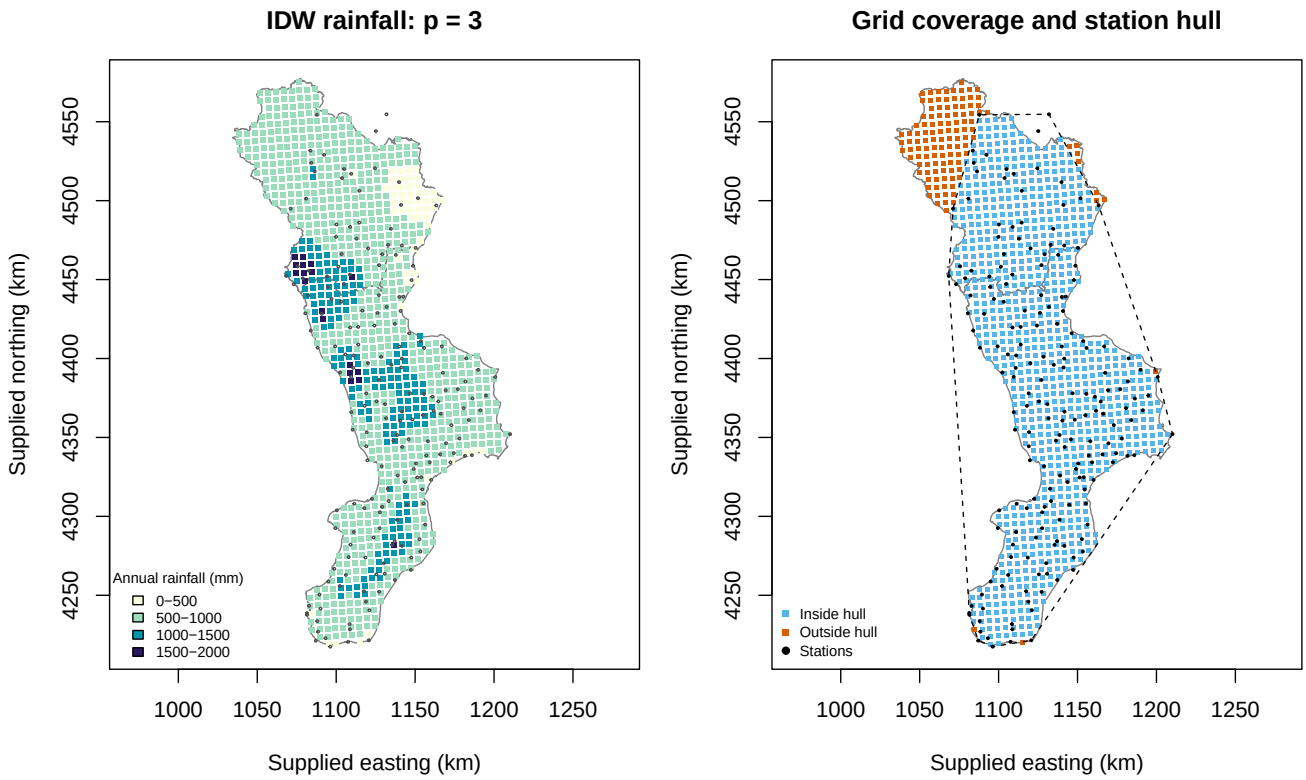
I now use all 189 stations and the power chosen from the full-data LOOCV. I predict at all 1,005 supplied land-grid locations. Their elevations remain in the result, but ordinary IDW uses coordinates and rainfall, not elevation. This is one limitation I noticed in the analysis: elevation is related to rainfall, but ordinary IDW does not use it directly.

```
prediction_grid$rain_idw <- idw_predict(dat, prediction_grid, best_p)
h <- chull(dat$x, dat$y)
prediction_grid$inside_hull <- sp::point.in.polygon(
  prediction_grid$x, prediction_grid$y, dat$x[h], dat$y[h]
) > 0
prediction_grid$nearest_km <- vapply(seq_len(nrow(prediction_grid)),
  function(i) min(sqrt((dat$x - prediction_grid$x[i])^2 +
                       (dat$y - prediction_grid$y[i])^2)), numeric(1))
stopifnot(nrow(prediction_grid) == nrow(land_grid),
          all(is.finite(prediction_grid$rain_idw)),
          all(prediction_grid$rain_idw >= min(dat$rain) - 1e-8),
          all(prediction_grid$rain_idw <= max(dat$rain) + 1e-8))
grid_summary <- data.frame(
  Grid_points = nrow(prediction_grid),
  Outside_hull = sum(!prediction_grid$inside_hull),
  Min_prediction_mm = min(prediction_grid$rain_idw),
  Max_prediction_mm = max(prediction_grid$rain_idw),
```

```
  Max_nearest_distance_km = max(prediction_grid$nearest_km)
)
knitr::kable(grid_summary, digits = 2)
```

Grid_points	Outside_hull	Min_prediction_mm	Max_prediction_mm	Max_nearest_distance_km
1005	117	291.4	1923.08	46.44

```
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
map_frame(paste("IDW rainfall: p =", best_p))
points(prediction_grid$x, prediction_grid$y, pch = 15, cex = 0.6,
       col = rain_colour(prediction_grid$rain_idw))
points(dat$x, dat$y, pch = 1, cex = 0.25, col = "grey30")
rain_legend()
map_frame("Grid coverage and station hull")
points(prediction_grid$x, prediction_grid$y, pch = 15, cex = 0.5,
       col = ifelse(prediction_grid$inside_hull, "#56B4E9", "#D55E00"))
polygon(dat$x[h], dat$y[h], border = "black", lty = 2)
points(dat$x, dat$y, pch = 19, cex = 0.25)
legend("bottomleft", c("Inside hull", "Outside hull", "Stations"),
       col = c("#56B4E9", "#D55E00", "black"), pch = c(15, 15, 19),
       bty = "n", cex = 0.7)
```



I use the same colour breaks for the predicted and observed rainfall maps. Each square shows a supplied grid location. I leave the gaps between the points visible. My predicted rainfall ranges from **291.4 to 1923.1 mm**.

There are **117 grid points outside the station convex hull**, out of 1005. I retain and flag them. These points lie outside the area enclosed by the outermost stations, so I would be more careful with their predictions. A point inside this boundary can also be far from any station; being inside does not guarantee accuracy. The largest distance from a grid point to its nearest station is **46.4 km**.

The wettest predicted patches are local rather than a simple increase from south to north. The northern and western edges of the supplied grid include locations beyond the station envelope, where the map is less well supported.

There are no observed rainfall totals at these grid locations, so I cannot calculate an accuracy score for the map itself. The station cross-validation scores provide the available assessment. IDW also gives no model-based prediction intervals; the coverage map should not be interpreted as one.

5 Weak and good AI prompts

5.1 Weak prompt

> I have rainfall data to analyse in R. Show that the rainfall is spatially dependent, use IDW with p = 2, and draw a rainfall map on the grid.

AI response, recorded for this report:

> I would select one year, plot the station values and an empirical variogram, and use `gstat::idw` with `idp = 2` to predict at the grid coordinates. An increasing variogram can suggest spatial structure, but it does not by itself prove residual spatial dependence: a broad mean trend can also create this pattern. The coordinate system and the choice of grid need checking before interpolation. Using p = 2 gives a conventional starting choice, not a power selected by cross-validation.

My assessment: I think this answer is a useful starting point. It correctly says that a variogram alone does not prove spatial dependence. However, it does not give me code to run, choose p using the data, or fix the station label. My prompt also asks the AI to confirm a result instead of checking whether it is supported. I fixed p at 2 even though the homework asks me to choose it. In the calculation above, p = 2 has LOOCV RMSE 224.02 mm, while the selected p has 209.49 mm. This comparison uses the same observations and folds, but both numbers are tuning results, not separate test results.

5.2 Good prompt

> I have RainData.RData and AllGrids.RData for my rainfall homework in R. Please first check what is in the files, which years are available, and which grid matches the rainfall data. Check the units, missing values and repeated locations. Choose one year based on complete records and station coverage, rather than which year gives the lowest error, and explain why. One station labelled Puglia belongs to Calabria. Check its location, fix the label and keep its measurements. Draw a station map and help me examine how rainfall changes with elevation, location and region. Look at the distribution, simple trends and variograms before and after removing a trend. Show the distance groups and pair counts, and explain the limits of these checks. When comparing trends, consider both the prediction error and the nugget. Explain whether normality is needed for IDW. Use simple R code with gstat::idw to choose p by cross-validation. State which values of p are tested and which score chooses the best one. If you assess the chosen method, keep the test observations out of parameter selection. Make the rainfall map on the matching supplied grid and mark places where there are few nearby stations or where prediction extends beyond the station area. Only report numbers that were actually calculated. Put the code, plots and clear explanations together in an R Markdown report with a PDF version, and avoid conclusions stronger than the results support.

AI response, recorded for this report:

> The rainfall file contains observations from 1970–1982 and a matching land grid. The other file contains marine predictors for 1995–2010 and should not be silently joined to these rainfall observations. I will use the existing land grid in RainData.RData. The earliest year tied for the greatest number of complete records is 1970, with 189 stations. Station 4367 is the Puglia-labelled station inside the supplied Calabria outline; its label can be corrected without changing its measurement.
>
> Elevation is positively associated with rainfall, but a simple trend leaves substantial variation. A raw variogram can combine mean variation and residual dependence, so I will compare it with residual variograms and show pair counts. These plots do not prove stationarity. The residual nugget is model-dependent, and normal-looking residuals would not prove joint Gaussianity. IDW itself does not require

normality, so I will keep the final predictions on the original rainfall scale and label the log analysis as a diagnostic.

I will implement `idw_predict` and `choose_p` using gstat, leave each station out before predicting it, and choose the candidate power with minimum RMSE. An outer five-fold check will repeat tuning inside each training fold so that its held-out observations do not select p. I will then use all 1970 observations for the final land-grid predictions and flag points outside the station hull. Random-fold validation may be optimistic for distant unsampled areas, and there is no grid accuracy score without observed grid rainfall. The runnable implementation and calculated results are in the report above.

My assessment: I find this answer more useful because it checks the actual files and notices that one grid does not match the rainfall data. It separates a broad trend from dependence that remains after removing the trend. It also explains that a variogram does not prove stationarity and that IDW does not need normally distributed data. I can check how p is selected and see that the outer test observations are not used to choose it. These are the main improvements for me, rather than just getting a longer answer. I still have to remember that the units and coordinate system are not fully described in the files. A detailed prompt cannot provide information that is missing.

I include these AI responses from the same conversation used to prepare the report. The AI already had access to the data checks, so this is not a comparison of two independent conversations. I have kept the weak answer's useful points as well as its limits. The code above follows the good prompt's approach. The p = 2 result comes from the same comparison table, not from a separate earlier AI experiment.

6 What I learned

From this analysis, I learned that elevation helps explain rainfall differences, but a simple trend still leaves a lot unexplained. Cross-validation gives me a reason for choosing p rather than just using the default. I would be more careful with predictions far from the stations or outside their boundary. I have studied only one year, used IDW without elevation, and assessed it with one random five-fold split. These choices, along with the missing information about units and coordinates, limit how far I can extend my conclusions.

My use of AI. I used AI assistance for the prompts, analysis plan, R code and draft explanations. This also includes help with the wording of my discussion. The results in this report come from running the code. I remain responsible for checking the work and understanding the choices before I submit it.

The R help pages for the functions in my analysis are `help("idw", package = "gstat")`, `help("variogram", package = "gstat")`, `help("fit.variogram", package = "gstat")`, and `help("point.in.polygon", package = "sp")`. I use weighted least squares to fit the variogram curves. IDW then calculates weighted averages. I have not fitted a Bayesian or likelihood-based spatial model in this homework.

```
versions <- data.frame(
  Component = c("R", "gstat", "sp", "knitr", "rmarkdown"),
  Version = c(as.character(getRversion()),
    vapply(c("gstat", "sp", "knitr", "rmarkdown"),
           function(p) as.character(packageVersion(p)), character(1)))
)
knitr::kable(versions, row.names = FALSE)
```

Component	Version
R	4.5.0
gstat	2.1.6
sp	2.2.0
knitr	1.50
rmarkdown	2.29